VERA

Vulnerability Evidence and Reasoning Agent

Evidence-based security analysis with a local model and an inspectable workflow.

Workflow
One stateful LangGraph

Routes
Five acquisition paths

Model
Local and bounded

State
SQLite + Kuzu

The model, API and explorers run on the local machine.

Alerts and unsupported claims

01 · signal

A port scan is reconnaissance

It shows scanning activity, but not whether a host was compromised.

02 · fluency

Fluent output without support

A model answer may be fluent while its citations are missing or irrelevant.

03 · proof

Claims are checked against evidence

The validator checks identifiers, numbers and quotations against fetched evidence.

> *Assessments report evidence, confidence and completeness.*

Detect	Retrieve	Validate	Review	Report
signal from events	local + graph evidence	citations checked	human review for high risk	assessment, limits, sources

What VERA is

VERA is a teaching prototype for evidence-based security analysis: one LangGraph workflow over curated knowledge, local retrieval, a knowledge graph and event rules.

01

Quoted evidence

Supported facts quote non-memory evidence verbatim.

02

Bounded model use

Deterministic checks verify which statements are supported.

03

Local execution

The model, API and explorers run locally.

04

Visible workflow

The UI exposes routes, evidence, warnings and reviews.

In scope: one workflow, five routes, a local model, a trace.

Out of scope: agent swarms, cloud dependency, asset inventory.

Architecture and repository layout

Modules are separated by typed contracts.

domain
Request, evidence, claim and result contracts.

routing
Intent rules; optional LLM disambiguation.

retrieval
Ranking, embeddings, corpus sync.

sources
Read-only CVE.org, NVD, KEV, EPSS, OSV.

stream
Events, PCAP, detectors, statistics.

providers
Deterministic, OpenAI-compatible, Ollama, Gemini.

reasoning
Citation validation, repair, reflection.

memory
SQLite history, memories, review ledger.

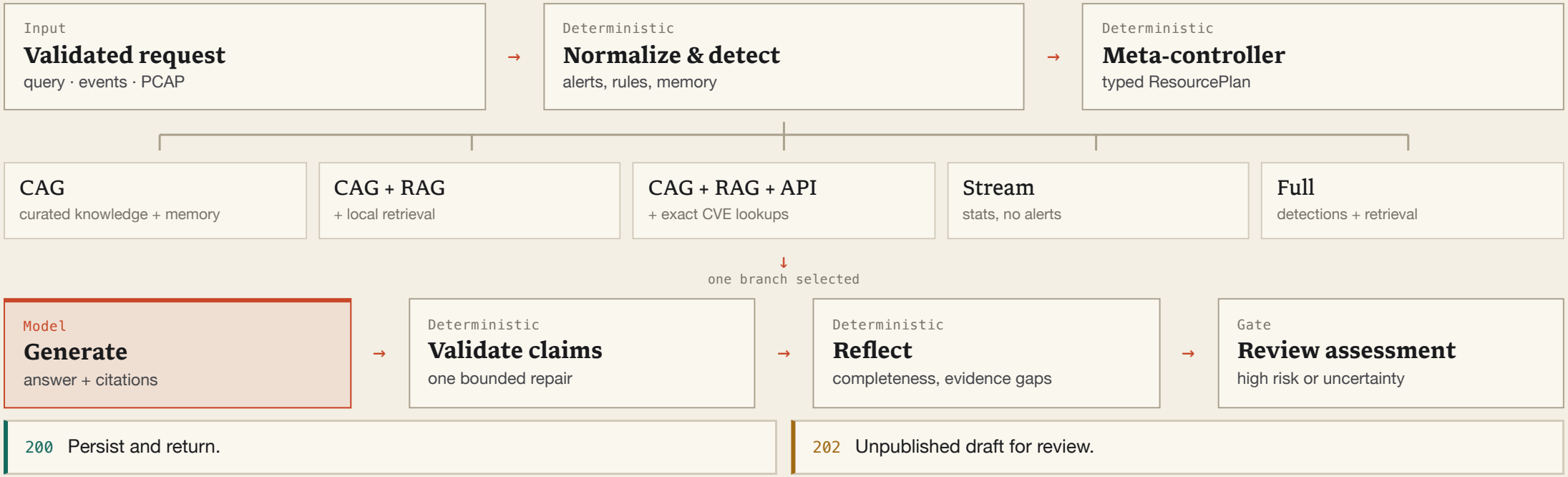
workflow
LangGraph topology and wiring.

interfaces
FastAPI API and Typer CLI.

```
src/vera/ → domain · workflow · retrieval · sources · stream · providers · reasoning · memory · interfaces · data  |  tests/ → unit · integration · architecture
```

Workflow and acquisition paths

deterministic model involvement One resource plan selects one branch.



Context acquisition always runs between routing and generation.

Routing: four independent flags

Four independent flags. The `Route` label is derived; mixed requests combine resources.

Independent flags

CAG	Curated knowledge + memory.
RAG	Local corpus; optional Kuzu.
TOOL	Read-only CVE lookups.
STREAM	Events and detectors.

CAG-only applies when curated knowledge covers the topic. Live lookups need explicitly enabled sources.

Route	CAG	RAG	Tool	Stream	When
CAG	●	○	○	○	Conceptual question with curated coverage.
CAG + RAG	●	●	○	○	Needs documentary sources.
CAG + RAG + API	●	●	●	○	CVE IDs present; live sources enabled.
Stream	○	○	○	●	Events without alerts.
Full	●	●	●	●	Active alerts, e.g. `NET-PORT-SCAN`.

> One request can combine stream detection, retrieval and exact API lookups.

Model role and deterministic checks

Deterministic layer

- ✓ Routing and event detection
- ✓ BM25 + embedding ranking, provenance rerank
- ✓ Citation validation and lexical grounding
- ✓ Reflection on completeness and gaps
- ✓ Human-review triggers
- ✓ Graph projection and persistence

Without a model, the **deterministic** provider still runs the workflow.

Model tasks

- Resolve routing ambiguity (optional)
- Generate the answer and its citations
- Make one correction after an invalid claim
- Choose a ReAct action (optional, bounded)

No graph writes, arbitrary queries, URLs or commands. Memory cannot establish a fact.

Call budget

2 calls by default; `VERA_MAX_REACT_STEPS + 3` with ReAct.

Providers

Deterministic, OpenAI-compatible, Ollama, Unsloth, Gemini.

Explicit degradation

`provider_fallback`, `warnings`, model metadata.

How claims are validated

Claim types

supported_fact
A verbatim quotation from non-memory evidence.

system_inference
A structured mapping; still validated.

unavailable
Missing evidence, stated explicitly.

simulated
Synthetic data, labelled end to end.

What the validator checks

01 **Registry integrity**
Unknown IDs cannot support a fact.

02 **Quotation presence**
The quote must appear in the evidence.

03 **Coverage**
Terms, IDs, numbers and negation.

04 **Contradictions**
Irrelevant citations and simple contradictions are rejected.

05 **Bounded repair**
One retry, then an explicit inference.

> The validator performs **lexical checks** against grounded evidence. Coverage is computed from grounded results.

Hybrid retrieval and ranking

Hybrid score



Normalized scores. Exact IDs stay single tokens.

Above the relevance threshold

Candidates	Rerank	Dedupe	Top-k
bounded set	provenance · date · novelty	near-identical documents	selected evidence

- **Hash embeddings** by default; Sentence Transformers optional.
- **Ranking order:** live records, then synchronized feeds, then snapshots.
- Undated guidance gets a **neutral freshness** score.

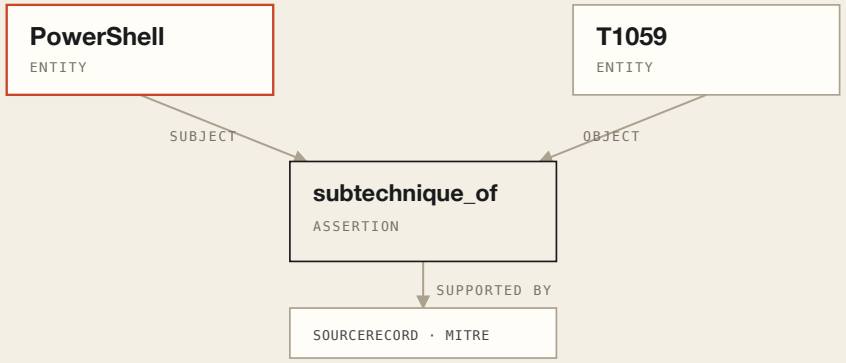
Quality signals exposed per result

Field	Meaning
`retrieval_score`	Original hybrid score
`quality_score`	Source quality
`freshness_score`	Freshness; neutral when undated
`novelty_score`	Novelty against the rest
`selection_score`	Final selection score

> **Explicit sync.** Allowlisted ATT&CK STIX and CISA KEV only; validated and staged before publishing.

Scores rank evidence; they do not verify source accuracy.

Knowledge graph: Kuzu



120
entities

112
supported assertions

ATT&CK 17.1 | CWE 4.20 | CAPEC 3.9 | KEV 2026.10.08 | OWASP 2021 | NIST

Node	Meaning
`Entity`	CVE, CWE, CAPEC, technique, tactic, mitigation, group, software, concept
`Assertion`	A sourced statement: subject + predicate + object
`SourceRecord`	Immutable evidence version with title, URL, status and metadata
`Run · Alert · Event`	Technical projection of the analysis history

`subtechnique_of` | `has_weakness` | `part_of` | `mitigates` | `uses` | `child_of` | `exploits_weakness` | `covers`

Traversal bounds

Hops
one or two logical hops; default one

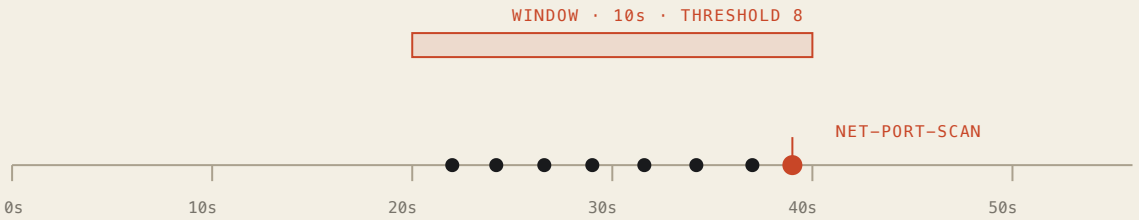
Results
12 per frontier, 8 identifiers max

Timeout
3,000 ms per query

Queries
fixed Cypher templates, bound inputs

> No arbitrary queries or graph writes. If Kuzu is missing, retrieval continues with **warnings**.

Stream detection and PCAP input



Alerts expose count, threshold, timestamps, duration and confidence.

Bundled synthetic sample

```
vera analyze "Analyze this traffic" \
  --events src/vera/data/samples/port-scan.jsonl \
  --simulated --offline --json
```

`benign-web` → `STREAM`, no alerts. `port-scan` → `FULL`, `NET-PORT-SCAN`.

Sliding windows

Port scans, auth failures and SYN bursts use configurable windows.

Event-local signatures

HTTP payload signatures stay event-local.

Restricted PCAP

PCAP is parsed from an argument list without a shell. Packet bytes stay outside model context.

Input rules

- ✓ Events via API or `--events`; raw logs in `query` are ignored.
- ✓ PCAP needs the CLI and `tshark`.
- ✓ `--simulated` labels synthetic data; `--offline` disables live lookups.

When human review is required

Deterministic review triggers

- High / critical alert
- Unsupported claim
- Missing usable evidence
- Out-of-scope conflict

Deterministic

Reflect

completeness and evidence gaps

→

Gate

Review assessment

ordinary path returns 200

200 · approve

Saved report

Persisted as a completed run.

200 · reject

Closed review

Nothing published; feedback required.

202 · revise

Pauses again

One model run, then pauses. Max two cycles.

Unpublished: drafts never appear in `/v1/runs`, history, memory or graph projection.

Local decisions: loopback and Origin checks; no identities. The CLI uses the same review gate.

Optional ReAct mode

Optional ReAct loop · `VERA_REACT_ENABLED=true`

Model decision

react_decide

typed action + short operational reason

↓

Deterministic tool

react_execute

local search · exact CVE lookup

↓

Observation

New evidence in the registry

back to decide

↺ until finish or budget

Exit

finish → Generate

generation resumes

Budget

2 calls default. ReAct: `steps + 3`. 3 steps → 6 calls; 10 → 13.

- Local context is tried first; tools and full mode defer live lookups.
- Typed decisions: corpus search and exact CVE lookups.
- Every decision needs a **short reason**.
- Reasons and outcomes land in the persisted trace.
- The loop cannot spend the two generation calls.

> The trace records decisions and outcomes, not hidden reasoning.

> **No web search**, arbitrary URLs or commands.

Safety boundaries

Shell-free PCAP

Argument lists for PCAP; fixed read-only HTTP for tools.

Packet data stays local

Packet bytes are parsed locally; evidence is treated as untrusted data.

Bounded work

Input, PCAP, model calls, tool calls and timeouts have limits.

Verbatim evidence

Unknown IDs cannot support a fact; quotes are required.

Explicit missing data

Missing data is recorded as **unavailable** evidence.

Loopback-only server

Loopback only; foreign clients and Origins are rejected.

Missing evidence is recorded as unavailable and remains citable.

Design rule

Persistence and recovery

Written by a completed run

01 **Run history**
Requests, evidence, reflection.

02 **Semantic memory**
Supported findings used only as context.

03 **Episodic memory**
Request, route, outcome.

04 **Checkpoints + ledger**
Drafts and versioned review decisions.

05 **Graph projection**
Committed with history; idempotent retries.

Recovery, in order

- `interrupt()` after reflection.
- `Command(resume=...)` continues the graph.
- Compare-and-set decisions with idempotency keys.
- Startup reconciles; missing state fails closed.

`review`
Max two revision cycles.

`conflict`
Stale version → `409`.

`pruning`
Terminal threads pruned.

`server`
One process per database.

CLI: `vera reviews` · `vera review <run_id> --approve`

> Drafts **never** appear in `/v1/runs`, history, memory or graph projection.

Interfaces

HTTP API `/docs`	`POST /v1/analyze` → report (`200`) or draft (`202`). Read runs, topology, reviews and health.	`/v1/analyze · /v1/runs · /v1/graph · /v1/reviews`
Typer CLI `vera`	The CLI exposes the same review gate without a server: analyze, serve, history, evaluate, import, review.	`--events · --pcap · --offline · --simulated · --json`
Workflow Explorer `/graph`	Compiled topology, previews, recorded runs, contracts and Mermaid. Previews never call a model.	`topology · preview · execution`
Knowledge Graph Explorer `/knowledge-graph`	Search by name, alias or ID; inspect neighbours, predicates, sources and provenance.	`entities · predicates · sources`

All components are served locally; there is no external renderer.

Local model setup

NVIDIA · 6 GB VRAM

Qwen3.5-4B

GGUF Q4_K_M via LM Studio CUDA, served as `vera-qwen3.5-4b`. 8,192 context, thinking off.

Apple silicon

Qwen3.5-4B MTP

GGUF Q6_K via `llama-server` with MTP. About **70% faster** decoding, same local quality.

Model files are **not** in the repository.

Verify the connection

- ✓ Model ID matches.
- ✓ `provider_fallback=false`: model answered.
- ✓ Nonempty `cited_evidence_ids`: graph evidence cited.
- ✓ `/health` is configuration, not inference.

Configuration

```
VERA_LLM_PROVIDER=openai
VERA_LLM_MODEL=Qwen3.5-4B
VERA_LLM_BASE_URL=http://127.0.0.1:1234/v1
VERA_KG_ENABLED=true
VERA_RETRIEVAL_K=3
VERA_MEMORY_K=1
VERA_REACT_ENABLED=false
VERA_LIVE_SOURCES=false
```

`model server :1234` → `VERA :8000` → `loopback browser`

> **No LLM?** `VERA_LLM_PROVIDER=deterministic` still runs.

Separate processes: `vera serve` does not start Qwen.

Code size and tests

7,557
lines of Python source

3,215
lines of test code

107
source modules

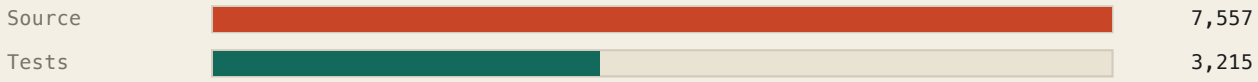
33
test modules

Test layout

- ✓ **unit/** — isolated domain behaviour.
- ✓ **integration/** — workflow, API, UI, persistence, evaluation.
- ✓ **architecture/** — repository-wide structural constraints.

Graph tests cover imports, two-hop retrieval, conflicts, rollback, missing databases, simulation and projection retries.

Lines of code



Test code is ~43% of the source line count.

Development commands

```
E="uv run --extra dev --extra knowledge-graph"

$E ruff check .
$E pytest
$E vera evaluate --json
$E vera corpus-sync
$E vera kg-build
```

Offline evaluation disables review to measure routing and grounding.

Dependencies

Runtime

- **Python 3.11+** · packages managed with `uv`
- **LangGraph** `≥0.2.60,<2` — workflow orchestration
- **langgraph-checkpoint-sqlite** `≥3.1,<4` — checkpoints
- **FastAPI** `≥0.115,<1` · **uvicorn** `≥0.34,<1`
- **Pydantic** `≥2.10,<3` · **pydantic-settings** `≥2.7,<3`
- **Typer** `≥0.15,<1` — CLI
- **httpx** `≥0.27,<1` — HTTP client

Versions are declared in `pyproject.toml` and locked by `uv.lock`.

Optional extras

- **kuzu** `==0.11.3` — `knowledge-graph` extra, pinned
- **sentence-transformers** `≥3.3,<6` — `embeddings` extra
- **pytest** `≥8.3,<10` · **pytest-cov** `≥6,<8` — `dev` extra
- **ruff** `≥0.9,<1` — lint (E, F, I, UP, B, SIM)

> Kuzu's upstream repository is **archived**: the dependency is pinned to 0.11.3 and the integration requires no extensions.

Not included: cloud SDK, external vector database, graph server, frontend build step.

Getting started

1 **Install**

```
uv sync --extra dev --extra knowledge-graph
cp -n .env.example .env
```

2 **Start a local model on port 1234**

```
# Bionic / LM Studio: load Qwen3.5-4B, serve :1234
curl -fsS http://127.0.0.1:1234/v1/models
```

Or, on Linux/NVIDIA: `bash scripts/start-local-model.sh`.

3 **Configure**

Set provider, exact model ID, base URL and `VERA_KG_ENABLED=true`. Restart after `.env` changes.

4 **Build the graph and serve**

```
uv run --extra knowledge-graph vera kg-build
uv run --extra knowledge-graph vera kg-status
uv run --extra knowledge-graph vera serve --open-browser
```

5 **Ask the first question**

```
curl -fsS http://127.0.0.1:8000/v1/analyze \
  -H 'Content-Type: application/json' \
  -d '{"query":"Explain PowerShell. Give a brief sourced claim."}'
```

Model server on `:1234`, VERA on `:8000`. `vera serve` does not start Qwen.

Where to look

`/docs · Swagger` `/graph · workflow`

`/knowledge-graph · domain` `/health · configuration`

No model? `VERA_LLM_PROVIDER=deterministic` still runs the workflow.

Check `metadata.model` and `provider_fallback=false`.

Limitations

No asset inventory
No version matching; no automatic CVE-to-technique mapping.

Dated corpus
The corpus is a focused snapshot and is not continuously refreshed.

Lexical grounding
Citations are checked lexically; full inference is out of scope.

Coverage score
The score measures evidence coverage. It does not measure whether conclusions are correct.

No expiry policy
Cached API records are not live lookups.

Limited concurrency
One API worker; close the database before replacing it.

Graph relations describe domain facts, not host compromise: PowerShell → subtechnique_of → T1059.

Teaching prototype only.